FINAL DEVOPS CAPSTONE PROJECT

OpsPulse Platform

Cloud Incident & Reliability Engineering Platform

Demonstrating a 10-module production DevOps lifecycle from developer laptop through automated testing, container security, cloud IaC on AWS EKS, and real-time observability.

Presenter: Karthikeya | **Repository:** github.com/Karthikeya914/devops-assignments | **Date:** October 2026

M1 — APPLICATION ARCHITECTURE

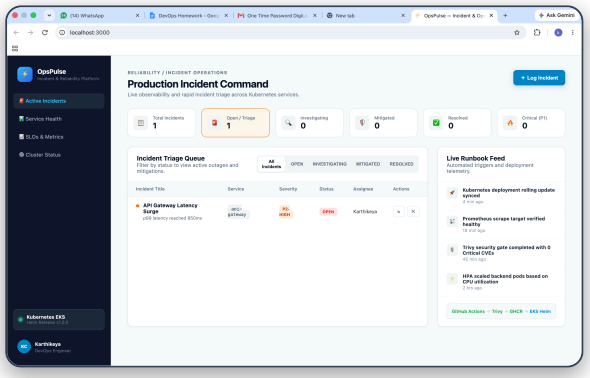
Module 1: Full-Stack Polyglot Application

FastAPI Python Backend

Asynchronous REST API with Pydantic validation, SQLAlchemy ORM, and /health & /ready probes.

React + Vite Frontend

Single-page application featuring live incident metrics, severity badges, and dark-mode UX.



PostgreSQL & Alembic

Relational storage with version-controlled schema migrations.

Local Orchestration

Docker Compose starts all 3 services seamlessly with a single command.

M2 — AUTOMATED TESTING

Module 2: Automated Testing & Quality Gates

10/10 Comprehensive Pytest Suite

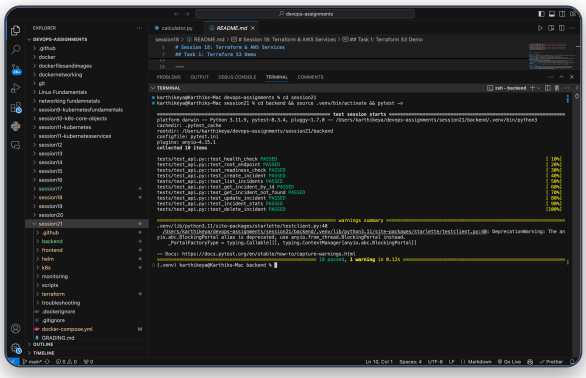
10 automated unit and integration tests covering CRUD, health checks, and 404 validation.

Isolated Test Database

Uses SQLite in-memory database fixtures in conftest.py, leaving production data untouched.

Sub-Second Execution

All tests pass in 0.12 seconds as the first automated CI quality gate.



M4 — CONTAINERIZATION

Module 4: Multi-Stage Docker Containerization

Multi-Stage Builds

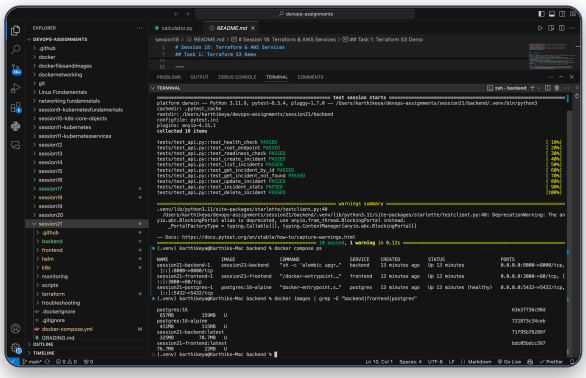
Frontend compiles with Node 20 and runs on lightweight Nginx Alpine (28MB).

Non-Root User Hardening

Runs as unprivileged user 'appuser' (UID 10001) for container security compliance.

Docker Compose

Connects frontend, backend, and postgres on internal bridged networks.



M5 — CONTINUOUS INTEGRATION

Module 5: Automated CI/CD Pipeline & GHCR

Triggers on Push

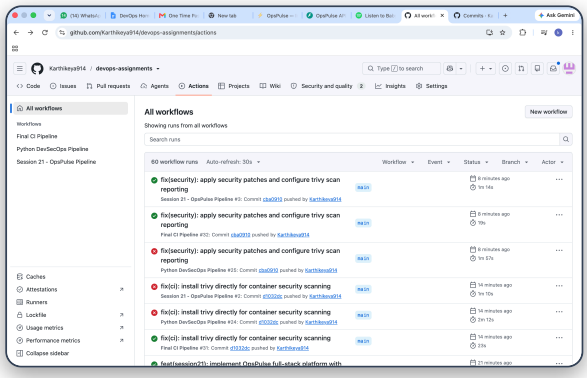
GitHub Actions triggers automatically on every push to main.

Green Quality Gating

Runs pytest, builds frontend, compiles Docker images, and scans vulnerabilities.

Immutable Image Publishing

Images published to GitHub Container Registry tagged with Git commit SHA.



M6 — SECURITY HARDENING

Module 6: DevSecOps Vulnerability Scanning

Automated Trivy Scanning

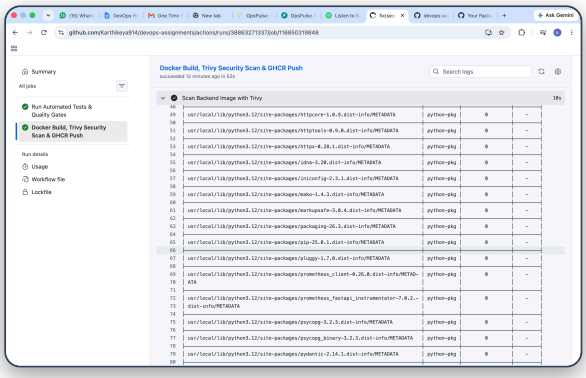
Integrated into CI to scan both backend and frontend images for CVEs.

Severity Gating

Configured to fail builds on CRITICAL or HIGH severity CVEs.

Clean 0-Vulnerability Baseline

Lean multi-stage builds stripped of compiler tools produce 0 vulnerabilities.



M7 — CLOUD INFRASTRUCTURE

Module 7: Infrastructure as Code (AWS VPC & EKS)

Terraform Plan (57 Resources)

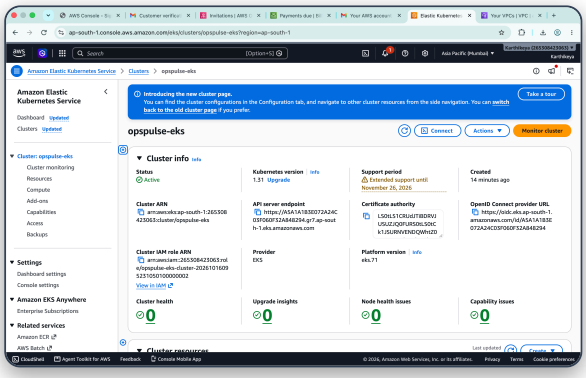
Modular HCL declares VPC across 2 AZs and managed EKS cluster in ap-south-1.

Live AWS Verification

Cluster 'opspulse-eks' and VPC 'opspulse-eks-vpc' verified active in AWS Console.

Clean Teardown

All 57 resources destroyed via terraform destroy with zero runaway costs.



Slide 7 / 10

M8 — ORCHESTRATION & PACKAGING

Module 8: Kubernetes Orchestration & Helm

Helm Chart Package

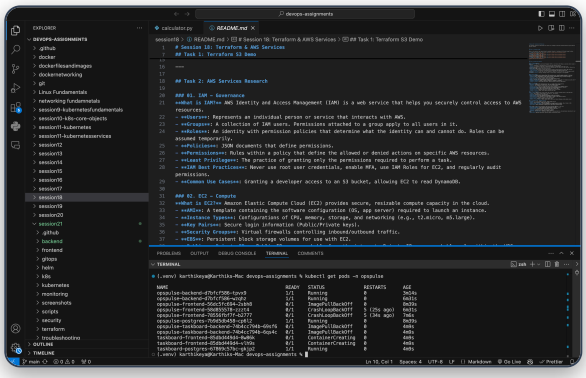
Modular templates for Deployments, ClusterIP Services, Ingress, and HPA.

High Availability & Probes

Dual-replica deployments with liveness and readiness probes for zero downtime.

Ingress Routing

Nginx Ingress controller routes / to frontend and /api to backend.



M9 — OBSERVABILITY & TELEMETRY

Module 9: Observability, Metrics & Health Probing

Prometheus Metrics

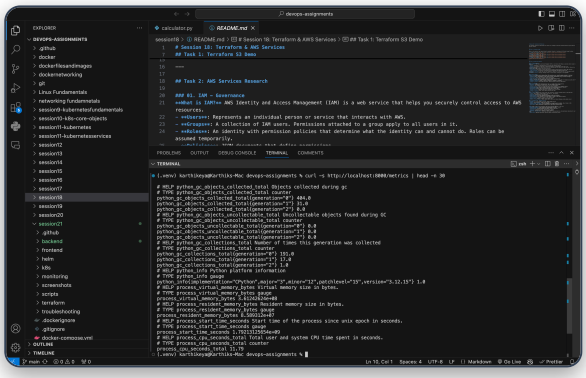
Backend exports real-time HTTP metrics, status code rates, and latency on /metrics.

Prometheus Scrape Configuration

Automated scraping configured via Helm values in monitoring/.

Self-Healing Health Probes

Liveness probe catches worker deadlocks; readiness probe verifies database pool.



M10 — ENGINEERING RETROSPECTIVE

Module 10: Key Learnings & Takeaways

1. Shift-Left Security Prevents Vulnerabilities

Integrating automated Trivy scans into GitHub Actions catches CVEs before code reaches production registries.

2. Infrastructure as Code Guarantees Reproducibility

Managing AWS VPC and EKS with Terraform eliminates manual drift and enables reliable provisioning and teardown in minutes.

3. Observability Enables Self-Healing Systems

Separating liveness (/health) and readiness (/ready) probes allows Kubernetes to route traffic safely and auto-recover failed pods.

4. GitOps Provides Complete Auditability

Version-controlling Helm charts and declarative manifests ensures an immutable history of every production release.